

BITS Pilani – Work Integrated Learning Programmes

M.Tech. Artificial Intelligence & Machine Learning
Course: AIMLCZG546 – Software Engineering for Machine Learning



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

**Work Integrated Learning
Programmes**

Assignment 1

Work Title

Predictive Maintenance in Mobile Hydraulic Systems

GitHub repository Link: [heavy-machinery-predictive-maintenance-pipeline](#)

Group No. 105

Team Contribution Details

Sl. No	BITS ID	Student Name	Contribution (Qualitative)	Contribution (%)
1	2024AC05744	Srinivasan R	System architecture design, lead the group, draft model implementation (Classification model), Model Registry implementation, architectural patterns, workflow design, code refactoring, documentation improvements and system integration.	100%
2	2024AC05100	Vineet Kumar	Data preprocessing pipeline, Docker-based deployment, infrastructure setup and CI support.	100%
3	2024AD05482	Vibhav Sharma	GR4ML requirements engineering (Business View, Analytics Design View, Data Preparation View and Quality Requirements), software architecture documentation, report preparation and project documentation.	100%
4	2024AC05064	Aman Kushwah	Machine learning model training and evaluation, Security Layer, Maintenance Advisor, FastAPI backend, React frontend, testing, deployment support and implementation documentation.	100%

Table of Contents

1. Introduction.....	4
2. Objective 1 – Requirements Formulation	5
2.1 Problem Statement	5
2.2 Requirements Formulation and Formulation of Measurable Goals using GR4ML	6
2.2.A. Overall Flow:	12
2.2.1 Business View (Why?).....	6
2.2.2 Analytics Design View (What?)	8
2.2.3 Data Preparation View (How?)	9
2.3 Top Three Quality Requirements	12
3. Objective 2 – Software Architecture and Implementation	14
3.1 System Context and Motivation.....	14
3.2 Building Block View: Architectural Patterns & Components	14
3.2.1 The Microservices Foundation.....	16
3.2.2 The Pipe-and-Filter Prediction Pipeline	18
3.3 Runtime View: End-to-End Application Workflow	21
3.4 Testing and Validation Strategy.....	22
3.4.1 Functional Testing.....	22
3.4.2 API Validation.....	23
3.4.3 User Interface Validation.....	25
3.4.4 Model Validation (Accuracy vs. Precision).....	27
3.5 Deployment Mechanism	28
3.5.1 Docker-based Deployment.....	28
3.5.2 Deployment Validation	30
3.5.3 Future Deployment Possibilities	30
4. Conclusion.....	33
5. References	33

1. Introduction

Industrial machines operating in construction, mining and manufacturing environments depend heavily on hydraulic systems for reliable operation. Failure of critical hydraulic components such as pumps, valves, coolers and accumulators can result in unplanned equipment downtime, increased maintenance cost and reduced productivity.

Conventional maintenance strategies generally rely on fixed service intervals or reactive maintenance after a component fails. These approaches do not always reflect the actual operating condition of the machine and may either replace healthy components unnecessarily or fail to prevent unexpected breakdowns.

This assignment presents a software system with ML for predictive maintenance of mobile hydraulic equipment using machine learning and software engineering principles. A detailed coverage starting from GR4ML, system architecture and execution perspective are explained.

A glimpse of how the application looks like is here as follows:

The screenshot shows a web browser window displaying the home page of a predictive maintenance application. The browser's address bar shows 'localhost:5173'. The page title is 'Predictive Maintenance — Mobile Hydraulics' with a subtitle 'Enter per-cycle hydraulic telemetry to assess component health and circuit stability.' The page is divided into two main sections: 'Sensor Readings' and 'Assessment'. The 'Sensor Readings' section is titled 'Per work-cycle machine telemetry' and contains a form for entering machine data. The 'Assessment' section is titled 'Component conditions - stability - repair guidance' and contains a button to 'Assess Machine'. The browser's taskbar at the bottom shows various open applications including AliExpress, Amazon.in, and Microsoft Teams.

Predictive Maintenance — Mobile Hydraulics
Enter per-cycle hydraulic telemetry to assess component health and circuit stability.

Sensor Readings
Per work-cycle machine telemetry

Machine type:

Operating hours: Pressure mean (bar):

Pressure std (bar): Flow mean (L/min):

Oil temp (°C): Vibration RMS (mm/s):

Motor power (kW): Pump speed (rpm):

Cooling efficiency (%):

Healthy preset | Degraded preset

Assess Machine

Assessment
Component conditions - stability - repair guidance

Enter readings and click Assess Machine.

Multi-output Random Forest - real-time stability model - RAG + OpenRouter LLM - secured API - Group 105

Figure 1.1: *[Home page of the deployed predictive maintenance web application.]*

2. Objective 1 – Requirements Formulation

2.1 Problem Statement

The Industry Challenge:

Heavy industrial equipment relies heavily on mobile hydraulic systems that operate under demanding conditions. Gradual wear and pressure loss in critical components—such as coolers, valves, pumps, and accumulators—frequently lead to unexpected failures, costly operational downtime, and elevated maintenance expenses.

Limitations of Current Approaches:

Traditional maintenance strategies fail to accurately reflect the real-time health of the machinery:

- **Preventive Maintenance (Scheduled):** Often results in the unnecessary and wasteful replacement of perfectly healthy components.
- **Reactive Maintenance (Run-to-Failure):** Leads to catastrophic breakdowns, production delays, and expensive emergency repairs.

Solution Approach:

To overcome the limitations of traditional maintenance, we engineered an end-to-end predictive maintenance system that integrates machine learning with robust software engineering:

- **Telemetry Processing:** Analyzes operational sensor data captured during each machine cycle to evaluate overall hydraulic circuit stability.
- **Component Diagnostics:** Predicts the real-time health of critical components, specifically targeting coolers, valves, pumps, and accumulators.
- **Actionable Guidance:** Translates ML predictions into targeted maintenance recommendations, empowering personnel to execute condition-based interventions before catastrophic failures occur.

2.2 Requirements Formulation and Formulation of Measurable Goals using GR4ML

2.2.1 Business View (Why?)

This view establishes the traceability from stakeholder needs to the specific insights the ML system must generate, ensuring alignment across organizational, product, and user levels. (refer next page for diagram).

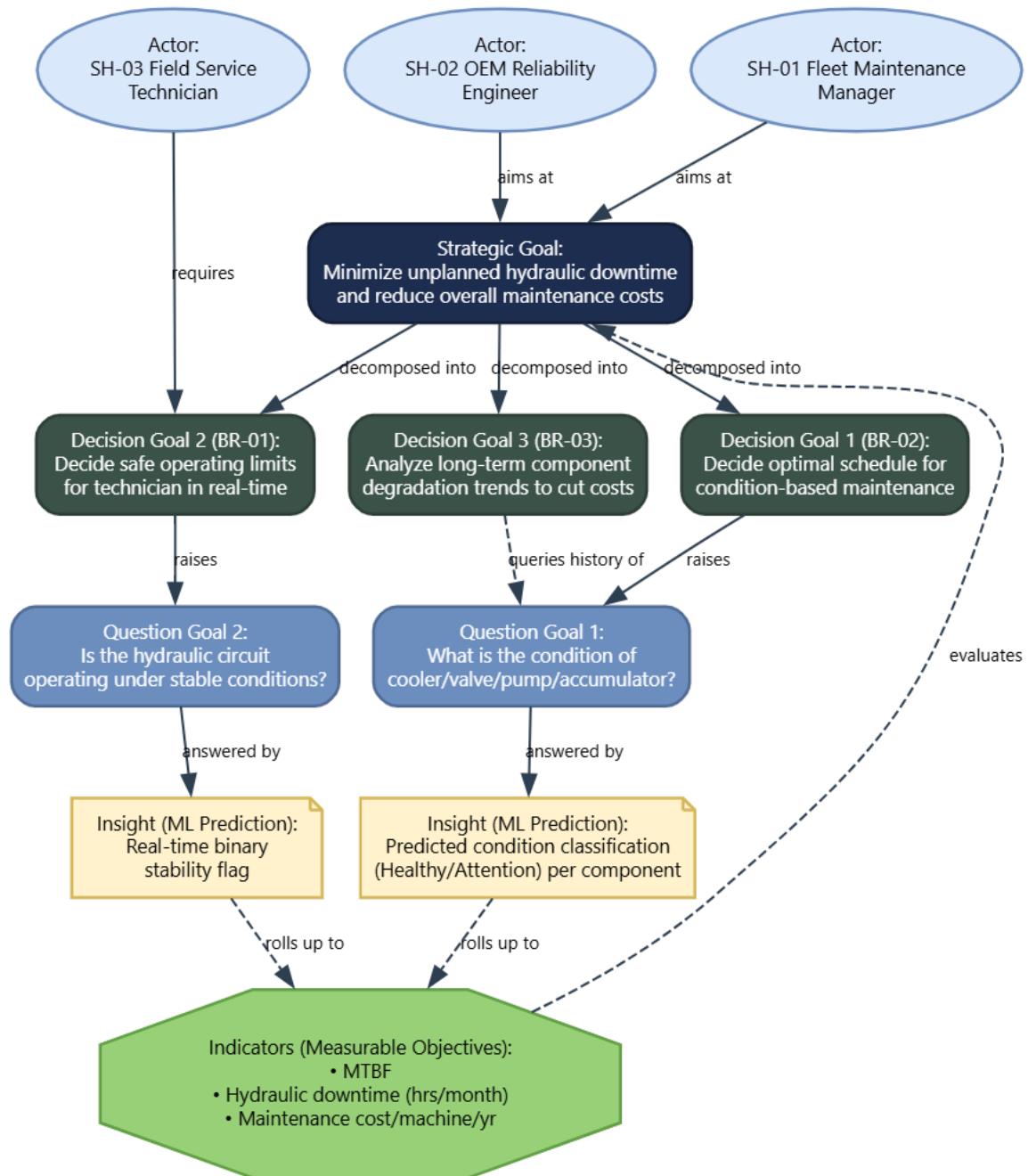


Figure 2.2: [GR4ML Business View diagram.]

List of Stakeholders:

- **SH-01: Fleet Maintenance Manager (Organizational/Product Focus)**
 - *Primary Concern:* Optimizing maintenance schedules to increase overall equipment availability.
- **SH-02: OEM Reliability Engineer (Organizational Focus)**
 - *Primary Concern:* Analyzing long-term component degradation trends to reduce overarching maintenance costs.
- **SH-03: Field Service Technician (User Focus)**
 - *Primary Concern:* Getting accurate, real-time diagnostic answers and completing repair tasks easily and safely.

Goal-to-Prediction Mapping Chain

Following the structured mapping of decisions routinely made within the organization to further their goals:

- **Strategic (Organizational) Goal:** Minimize unplanned hydraulic downtime and reduce overall maintenance costs.
 - *Supports Decision Goal 1 (Product Level):* Decide the optimal schedule for condition-based maintenance per machine.
 - *Informs Question Goal 1:* What will be the health condition of the cooler, valve, pump, and accumulator during the current operating cycle?
 - *Answers with Insight (ML Prediction):* Predicted condition classification (Healthy/Attention) for each component.
 - *Supports Decision Goal 2 (User Level):* Decide safe operating limits for the technician in real-time.
 - *Informs Question Goal 2:* Is the hydraulic circuit operating under stable conditions right now?
 - *Answers with Insight (ML Prediction):* Real-time binary stability flag.

Indicators (Measurable Objectives)

- Mean Time Between Failures (MTBF).
- Hydraulic downtime (hours/month).
- Maintenance cost per machine per year.

Formal Business Requirements (IREB Standard)

- **BR-01:** The system shall provide SH-03 with real-time diagnostic alerts regarding hydraulic circuit stability to support safe operational decisions.
- **BR-02:** The system shall generate actionable maintenance recommendations to enable SH-01 to schedule condition-based interventions.

- **BR-03:** The system shall log component degradation telemetry and prediction insights over time to enable SH-02 to analyze long-term reliability trends and evaluate fleet-wide MTBF metrics.

2.2.2 Analytics Design View (What?)

This view translates the Business View's operational questions into specific analytical objectives and evaluates algorithms based on required quality attributes.

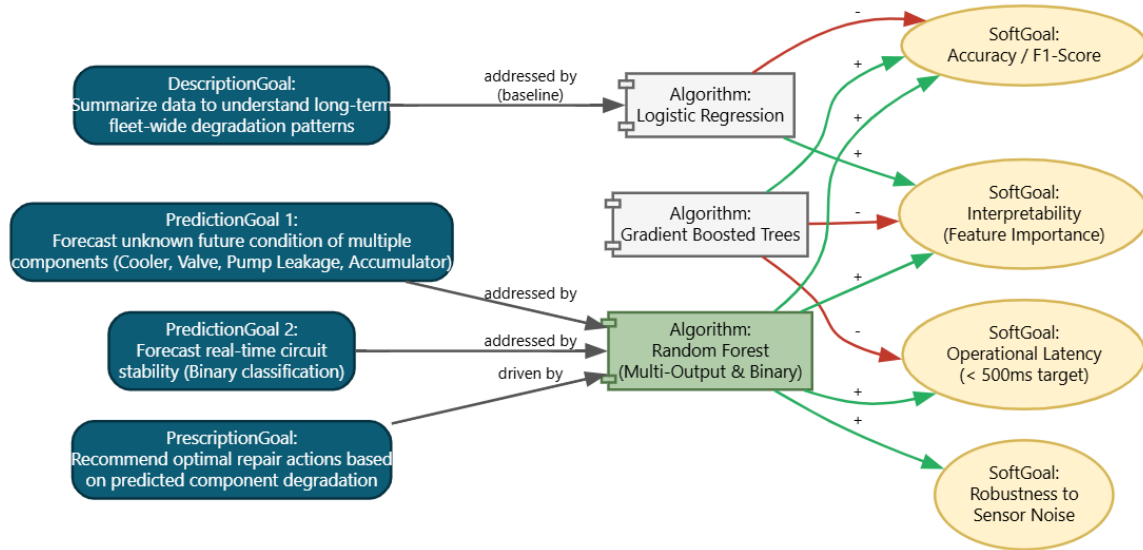


Figure 2.3: *[GR4ML Analytics Design View diagram.]*

Analytics Goals:

- **PredictionGoal 1:** Forecast the unknown future condition of multiple components (Cooler, Valve, Pump Leakage, Accumulator).
- **PredictionGoal 2:** Forecast real-time circuit stability (Binary classification).
- **DescriptionGoal:** Summarize data to understand long-term fleet-wide degradation patterns.
- **PrescriptionGoal:** Recommend optimal repair actions based on the predicted component degradation.

Model Goals (SoftGoals / Quality Attributes):

Soft Goal	Importance to the Project
Accuracy / F1-Score	Correctly identify degrading hydraulic components while minimising false predictions.
Interpretability (Feature Importance)	Help maintenance engineers understand which sensor measurements contributed most to a prediction.
Operational Latency (<500 ms target)	Enable near real-time predictions suitable for live operational use, accounting for full end-to-end network round-trip time.
Robustness to Sensor Noise	Maintain reliable predictions even when sensor readings contain measurement noise or minor variations.

- **Algorithm Selection & Influence:**
 - **Logistic Regression (Baseline):** Positively influences Interpretability, but negatively influences Accuracy on complex non-linear sensor data.
 - **Random Forest (Selected):** Positively influences Accuracy, Interpretability, and tolerance to missing values/noise, satisfying the hierarchical goals of the system.

Formal System Requirements (IREB Standard)

- **FR-01 (Functional):** The prediction engine shall classify the operating condition of the cooler, valve, pump, and accumulator using a Random Forest algorithm.
- **QR-01 (Performance):** The system shall execute the multi-output prediction inference along with suggestion to the user with a latency of strictly less than 500 milliseconds to satisfy the user goal of quick task completion.

2.2.3 Data Preparation View (How?)

This view defines the operations performed to transform raw data into a usable format, providing a clear reference for the data engineering pipeline.

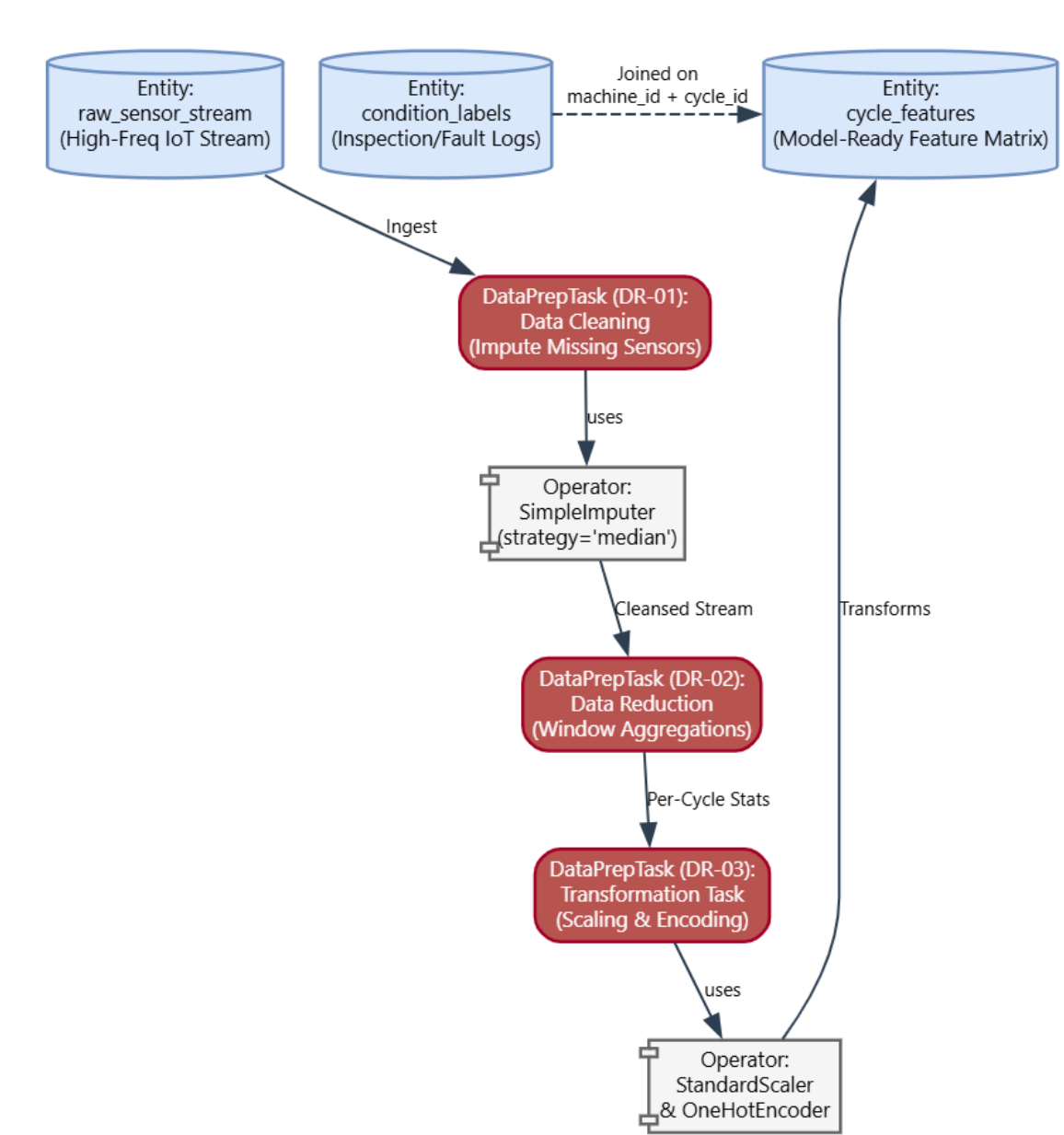


Figure 2.4: [GR4ML Data Preparation View diagram.]

Entities:

- raw_sensor_stream: High-frequency IoT data objects representing pressure, temperature, vibration, and flow.
- cycle_features: The transformed, model-ready feature matrix.

Data Preparation Tasks & Operators:

1. **Data Cleaning Task:** Removes missing values and inconsistencies from the data. (Operator: SimpleImputer using median mechanism).

2. **Data Reduction Task:** Reduces data dimensionality while preserving important per-cycle information. (Operator: Window aggregations).
3. **Transformation Task:** Applies an algorithm within preprocessing to scale variables. (Operator: StandardScaler & OneHotEncoder).

Formal Data Processing Requirements (IREB Standard)

- **DR-01:** The system shall execute a data cleaning task to impute missing values in the raw sensor stream using a median-based mechanism.
- **DR-02:** The system shall execute a data reduction task to extract statistical features over each distinct operating cycle.

2.3 Top Three Quality Requirements

To ensure the predictive maintenance system is practical and reliable for industrial deployment, the following top three quality requirements (non-functional requirements) have been prioritized and justified:

ID	Quality Attribute	Formal Requirement Specification (IREB Standard)	Justification for Selection
QR-01	Explainability	The system shall generate context-aware maintenance recommendations alongside any predicted component faults.	Maintenance personnel require actionable repair guidance to safely perform condition-based interventions, rather than relying on isolated machine learning probability labels.
QR-02	Performance (Latency)	The system shall complete the end-to-end request lifecycle—including ML inference and LLM-based guidance generation—and return the API response in strictly less than 500 milliseconds.	Real-time diagnostic alerts are mandatory for technicians to make immediate decisions. The 500 ms threshold accounts for network overhead (for LLM call roundtrip) while remaining imperceptible to the user during workflow execution.
QR-03	Robustness	The data preparation pipeline shall maintain valid feature extraction and prediction capabilities despite minor fluctuations or noise in the raw sensor stream.	Industrial hydraulic equipment operates in harsh environments; the system must remain computationally stable and avoid misclassification caused by standard mechanical vibration or sensor noise. <i>Note: While the current implementation is validated using a synthetic dataset, the data preparation pipeline (incorporating median imputation and standardization) is specifically architected to mitigate real-world sensor anomalies during future field deployments."</i>

2.4 Overall Flow

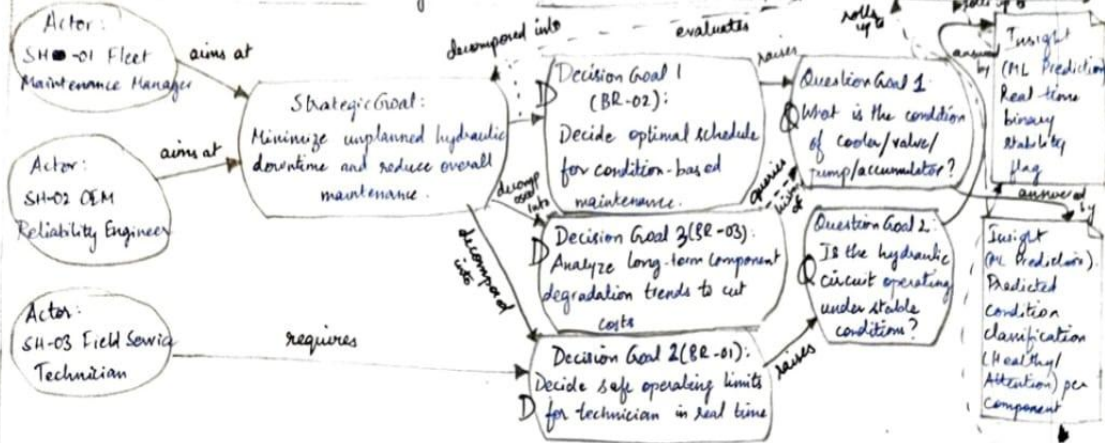
Please note that this section combines part 2 and 3 of the assignment topics. The overall GR4ML diagram is as follows

Figure 2.1: *[Overall GR4ML requirements engineering workflow adopted for the predictive maintenance system.]*

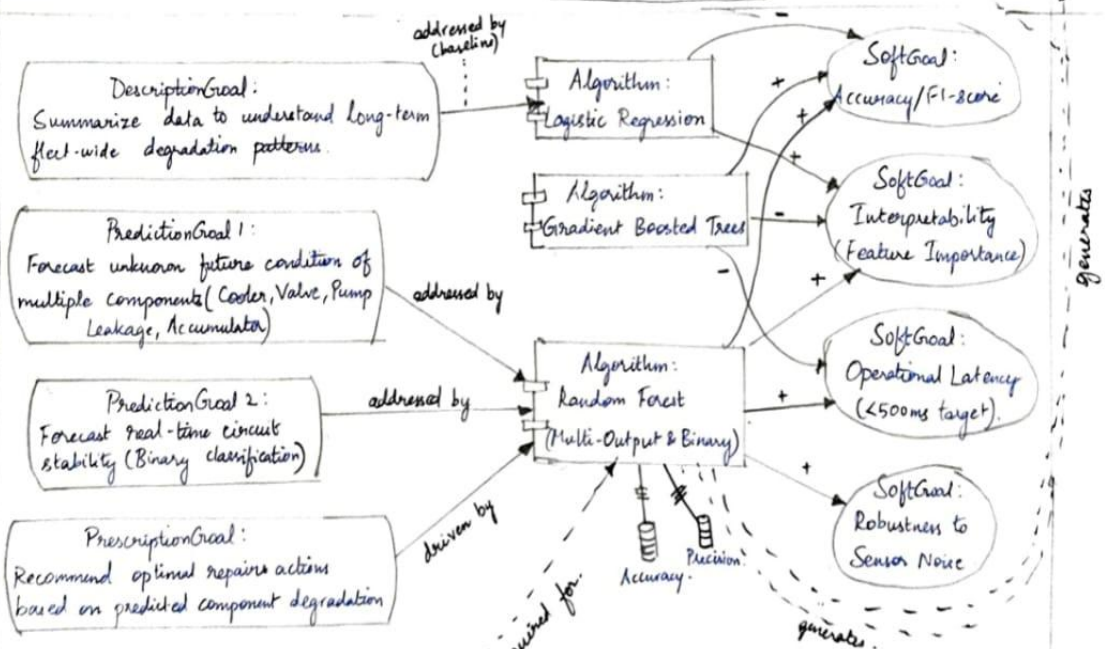
IRML - Predictive Maintenance for Mobile Hydraulics Machinery

- Putting it all together

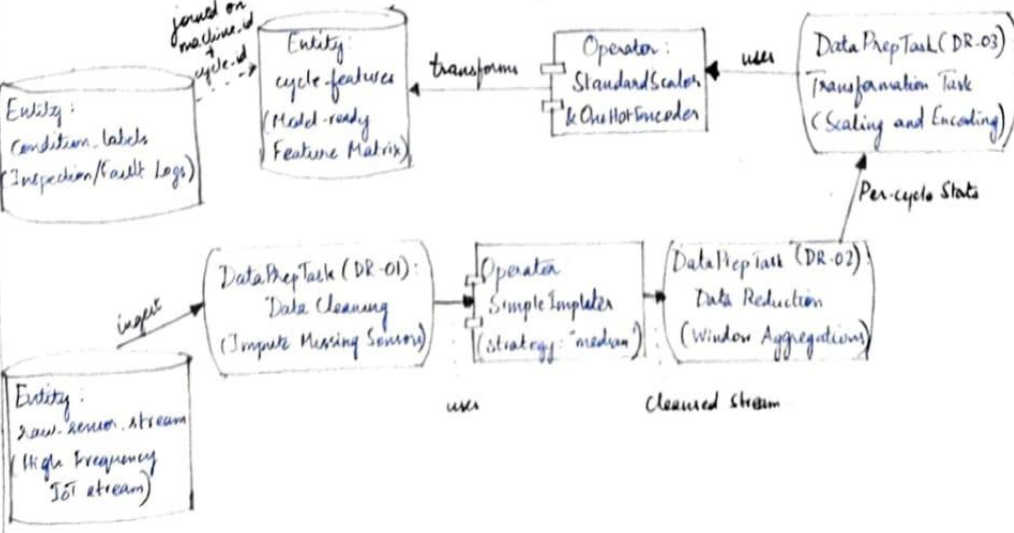
Business View



Analytics Design View



Data Preparation View



3. Objective 2 – Software Architecture and Implementation

3.1 System Context and Motivation

The requirements engineered in Objective 1 dictated a system that is not only highly accurate in its machine learning predictions but also robust and performant enough for operational use. To bridge the gap between a standalone predictive model and a deployable software product, the proposed solution was developed as a cohesive web application as an initial step. This section outlines the structural decisions, component interactions, and deployment strategies required to translate the GR4ML analytics design into a functional, end-to-end predictive maintenance system.

3.2 Building Block View: Architectural Patterns & Components

To ensure the system remains maintainable and scalable, the architecture avoids a monolithic design in favor of two complementary software patterns.

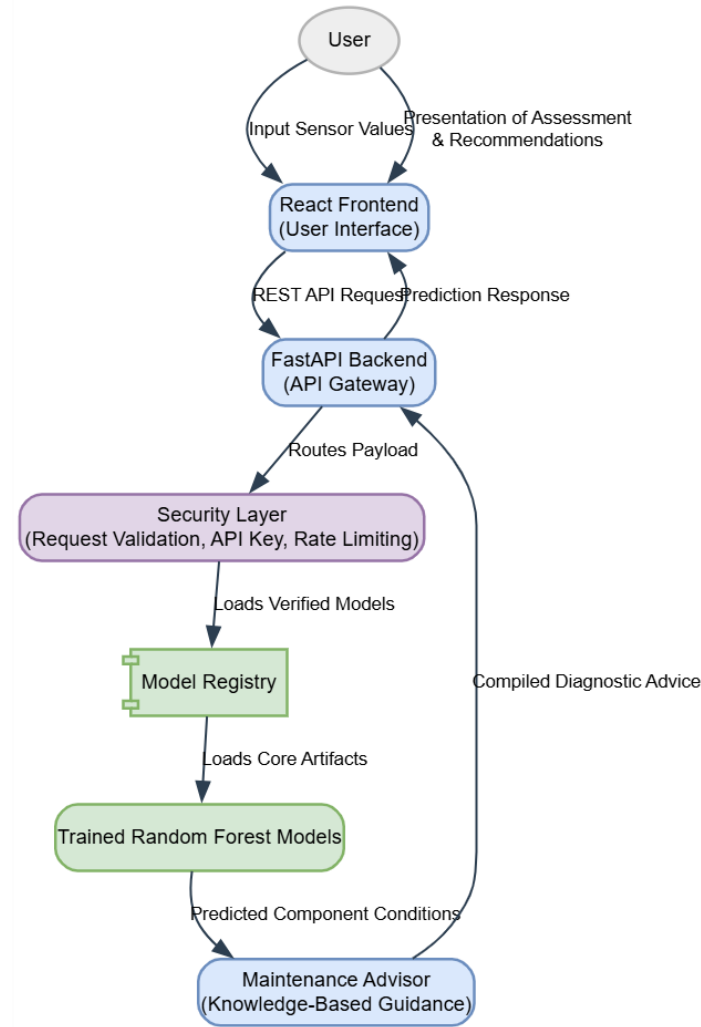


Figure 3.2: Overall software architecture of the predictive maintenance application showing both machine learning (ML) and non-machine learning (Non-ML) components.

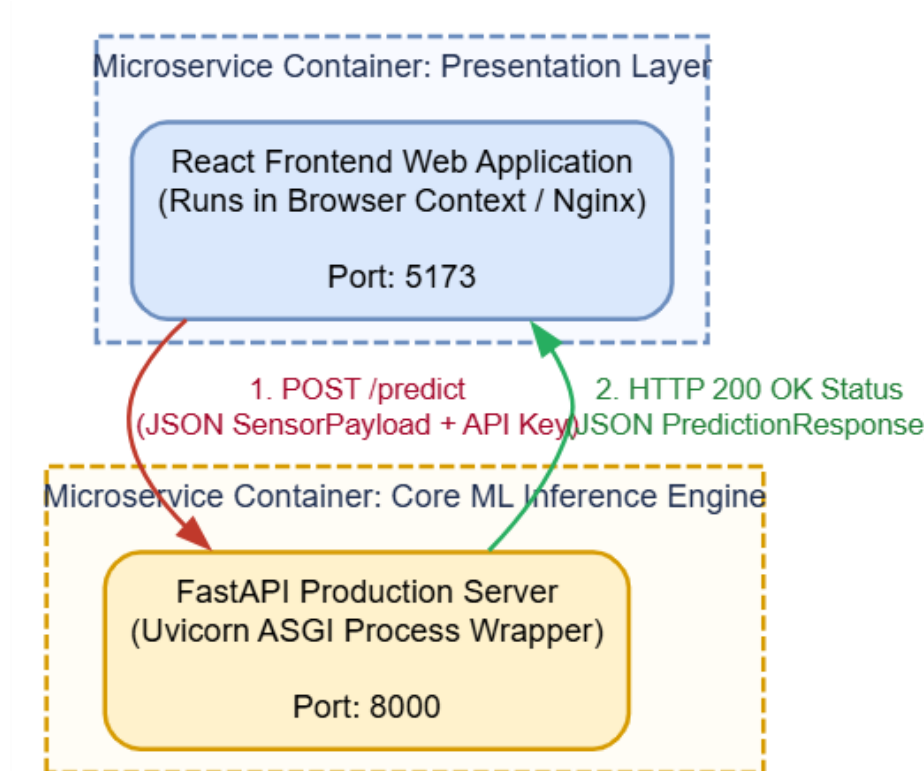
Component Classification

Component Category	Project Components
HW / Infrastructure	End-User Client Device (Workstation/Tablet), Network Router/Gateway
ML Components	Model Registry, Random Forest Models, RAG
SW Components	React Frontend Layer, FastAPI Backend Layer, Security Layer (API Validation Middleware)
Data Components	High-Frequency IoT Sensor Stream Payload (SensorPayload), Standardized Feature Matrix, System Prompts & Embeddings
Deployment Components	Docker Engine, docker-compose.yml Configuration, hydraulics_web Container, hydraulics_api Container

3.2.1 The Microservices Foundation

The application is bifurcated into two independently deployable services: a React-based frontend and a FastAPI backend.

- *Architectural Storyline:* By decoupling the presentation layer from the computationally heavy inference layer, the system gains operational flexibility. Maintenance personnel can interact with a lightweight UI, while the backend can independently scale or update its machine learning models without requiring full system downtime.



Figure

3.3: Microservices architecture showing communication between the React frontend and FastAPI backend.

Snippet: Docker Compose Microservices Orchestration

File: Docker_compose.yml

```
# One-command full stack for the Predictive-Maintenance app – Group 105.
```

```
#
```

```
# docker compose up --build
```

```
#
```

```
# Frontend -> http://localhost:5173      Backend/API ->
```

```
http://localhost:8000/docs
```

```
services:
```

```
  backend:
```

```
    build: .
```

```
    container_name: hydraulics_api
```



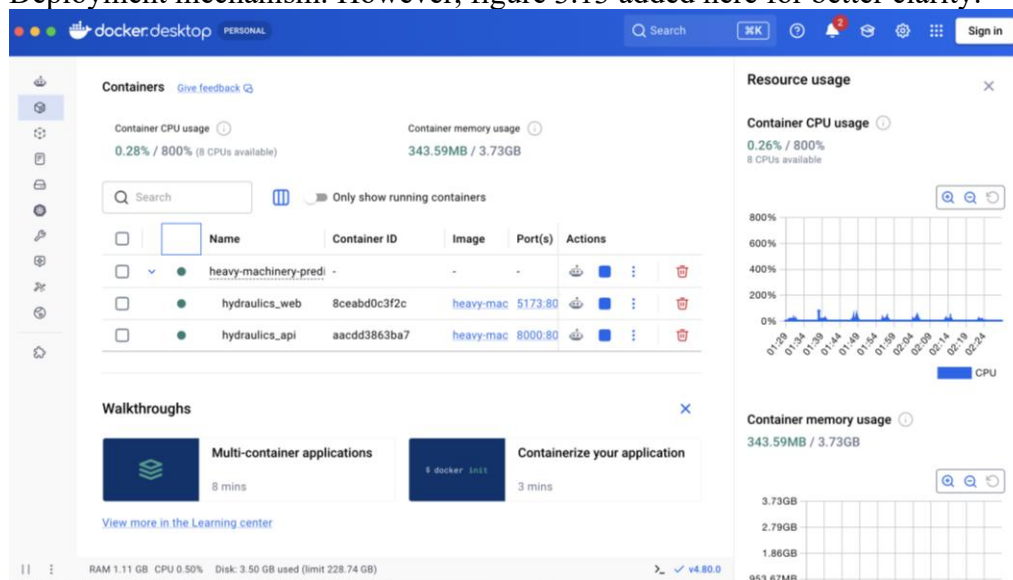
```

ports:
  - "8000:8000"
environment:
  # optional – enables the RAG advisor's LLM generation when set on
the host:
  # OPENROUTER_API_KEY=sk-or-... docker compose up
  - OPENROUTER_API_KEY=${OPENROUTER_API_KEY:-}
  - OPENROUTER_MODEL=${OPENROUTER_MODEL:-google/gemma-4-26b-a4b-
it:free}
healthcheck:
  test: ["CMD-SHELL", "python -c \"import urllib.request;
urllib.request.urlopen('http://localhost:8000/health')\""]
  interval: 10s
  timeout: 5s
  retries: 5

frontend:
  build: ./frontend
  container_name: hydraulics_web
  ports:
    - "5173:80"
  depends_on:
    - backend

```

Note: Screenshots for execution of `docker_compose.yml` are covered under Section 3.5 Deployment mechanism. However, figure 3.13 added here for better clarity.



3.2.2 The Pipe-and-Filter Prediction Pipeline

Within the backend microservice, the machine learning workflow is orchestrated using the Pipe-and-Filter pattern.

- *Architectural Storyline:* Rather than tangling request validation, feature processing, and inference into a single massive function, the data flows unidirectionally through isolated stages (Filters). A request passes from the **Security Layer** (which drops malformed or unauthorized payloads) into the **Prediction Module** (which accesses the Model Registry to execute the Random Forest inference), and finally to the **Maintenance Advisor** (which enriches the statistical prediction with LLM-generated repair guidance). This strict separation of concerns ensures that future enhancements will not break the API contract.

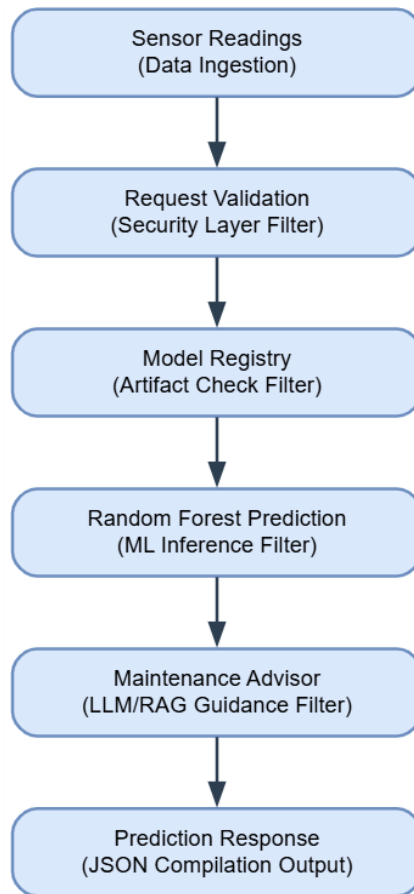


Figure 3.4: Pipe-and-Filter architecture implemented for the machine learning prediction pipeline.

Snippet 1: FastAPI Pipe-and-Filter Implementation

(from file `src/api/server.py` (where backend FastAPI app is set):

```
@app.post("/predict", response_model=PredictionResponse)
```

```

def predict(reading: SensorReading, x_api_key: str =
Header(default=API_KEY)):
    import time
    t0 = time.perf_counter()
    payload = reading.model_dump()
    machine_type = payload.pop("machine_type")

    # Filter 1 & 2: Security, Rate-Limiting, and Hydraulic Outlier
    Filtering
    try:
        def predict_fn(clean_payload):
            return clean_payload

        clean = _gateway.guarded_predict(
            client_id="frontend",
            api_key=x_api_key,
            payload=payload,
            predict_fn=predict_fn,
            bounds=HYDRAULIC_BOUNDS,
        )
    except SecurityError as e:
        raise HTTPException(status_code=400, detail=str(e))

    clean["machine_type"] = machine_type
    X = pd.DataFrame([[clean[c] for c in FEATURES]], columns=FEATURES)

    # Filter 3 & 4: Model Registry Validation & Machine Learning Inference
    cond_pred = _condition_model.predict(X)[0]
    stability = "unstable" if int(_stability_model.predict(X)[0]) == 1
else "stable"
    latency = round((time.perf_counter() - t0) * 1000, 2)

    # Filter 5: Maintenance Advisor Severity Resolution & RAG Enrichment
    components, worst = [], None
    worst_sev = 0
    for i, target in enumerate(TARGETS):
        cls = int(cond_pred[i])
        healthy = cls == HEALTHY[target]
        components.append(ComponentResult(component=target,
predicted_class=cls,
                                                    status="healthy" if healthy else
"attention"))
        sev = SEVERITY.get(target, {}).get(cls, 0)
        if not healthy and sev > worst_sev:
            worst_sev, worst = sev, target

```

```

procedure = guidance = llm = None
if worst:
    advice = _advisor.advise(worst, k=1)[0]
    procedure, guidance = advice["procedure"], advice["guidance"]
    llm = advice.get("llm_recommendation")

_audit.record("frontend", "predict", f"stability={stability}
worst={worst}")

# Filter 6: Structured Prediction Response Compilation
return PredictionResponse(
    components=components, stability=stability,
    flagged_component=worst,
    repair_procedure=procedure, repair_guidance=guidance,
    llm_recommendation=llm, latency_ms=latency)

```

[Figure 3.11 pasted here for better clarity]

localhost:8000/docs#/default/predict_predict_post

New folder (1) Amazon.in New folder vibhavsharma18 - O... https://webapps.idbi... Inbox (8,240) - vibh... Life Insurance Corp... VibhavSharma-7

Request URL: http://localhost:8000/predict

Server response

Code: 200

Response body

```

{
  "machine_type": "Excavator",
  "components": [
    {
      "component": "cooler_condition",
      "predicted_class": 0,
      "status": "attention"
    },
    {
      "component": "valve_condition",
      "predicted_class": 0,
      "status": "attention"
    },
    {
      "component": "pump_leakage",
      "predicted_class": 0,
      "status": "healthy"
    },
    {
      "component": "accumulator_pressure",
      "predicted_class": 1,
      "status": "attention"
    }
  ],
  "stability": "stable",
  "flagged_component": "cooler_condition"
}

```

Response headers

```

Access-Control-Allow-Origin: *
Content-Length: 956
Content-Type: application/json
Date: Mon, 09 Jul 2024 20:01:29 GMT
Server: envoy

```

Responses

Code	Description	Links
200	Successful Response	No links

Media type:

Example Value

```

{
  "components": [
    {
      "component": "string",
      "predicted_class": "string",
      "status": "string"
    }
  ],
  "stability": "string",
  "flagged_component": "string",
  "repair_procedure": "string",
  "repair_guidance": "string",
  "llm_recommendation": "string",
  "latency_ms": "string"
}

```

Validation Error

Media type:

Example Value

```

{
  "detail": [
    {
      "loc": [
        "string",
        "string"
      ],
      "msg": "string",
      "type": "string"
    }
  ]
}

```

3.3 Runtime View: End-to-End Application Workflow

Understanding the static components is only half the picture; the runtime execution demonstrates how these building blocks collaborate during a live prediction request.

Figure 4.4 - End-to-End Application Workflow

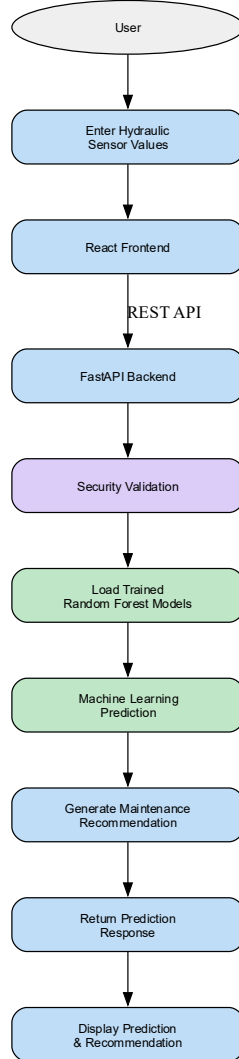


Figure 3.5: End-to-end execution workflow of the predictive maintenance application

When a field technician submits hydraulic telemetry via the React interface, a REST API payload is dispatched. The FastAPI backend intercepts this payload, first routing it through the Security Layer. Once validated, the system synthesizes the data. The Random Forest model classifies the health of the cooler, valve, pump, and accumulator. These classifications act as the context for the Maintenance Advisor, which translates the raw ML outputs into actionable, human-readable maintenance recommendations. Finally, the synthesized JSON response is routed back to the frontend for visualization.

3.4 Testing and Validation Strategy

Validating a machine learning application requires a dual-track approach: proving both the software reliability and the intelligence of the model itself.

3.4.1 Functional Testing

Functional testing was performed by executing complete prediction requests through the web application using representative hydraulic sensor measurements. The Healthy and Degraded preset configurations were used to verify that the application correctly processed different operating conditions and generated prediction results together with maintenance recommendations.

The testing confirmed that user input was successfully transmitted from the React frontend to the FastAPI backend, processed through the machine learning prediction pipeline and returned to the frontend without errors. The displayed prediction results were consistent with the selected operating conditions.

The screenshot displays a web application titled "Predictive Maintenance — Mobile Hydraulics" running on localhost:5173. The interface is divided into two main sections: "Sensor Readings" and "Assessment".

Sensor Readings: This section is titled "Per work-cycle machine telemetry". It includes a dropdown menu for "Machine type" set to "Excavator". Below this are several input fields for sensor data: "Operating hours" (300), "Pressure mean (bar)" (200), "Pressure std (bar)" (4), "Flow mean (L/min)" (8.3), "Oil temp (°C)" (50), "Vibration RMS (mm/s)" (1.5), "Motor power (kW)" (16), "Pump speed (rpm)" (1445), and "Cooling efficiency (%)" (95). At the bottom of this section are two radio buttons: "Healthy preset" (selected) and "Degraded preset", followed by an "Assess Machine" button.

Assessment: This section is titled "Component conditions · stability · repair guidance". It shows a green status indicator "Circuit STABLE" with a "Real-time stability model" label. Below this, a "Component conditions" section lists four items: "Cooler" (OK), "Valve" (OK), "Pump leakage" (OK), and "Accumulator" (OK). At the bottom of the assessment section, it states "Inference latency: 417.1 ms".

At the very bottom of the page, a small footer reads: "Multi-output Random Forest · real-time stability model · RAG + OpenRouter LLM · secured API · Group 105".

Figure 3.9: Healthy preset prediction result

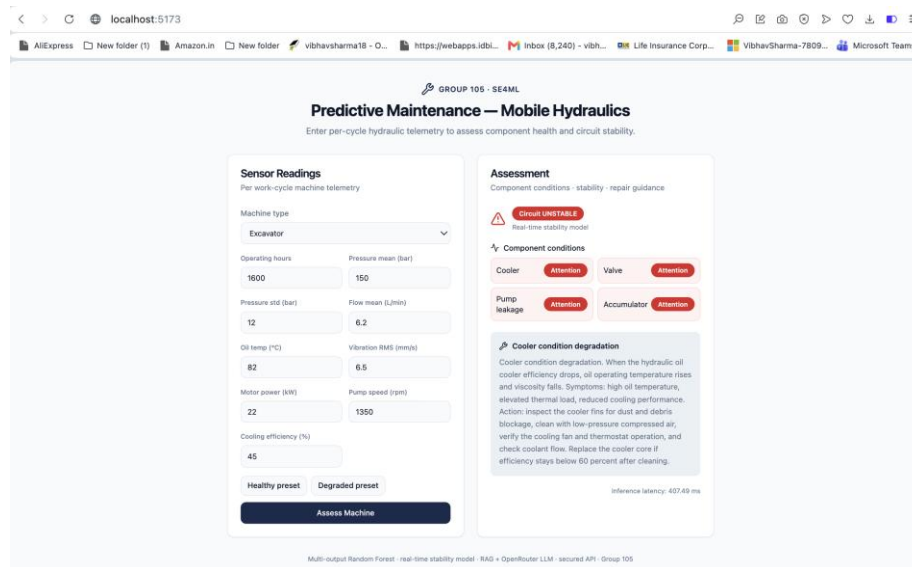
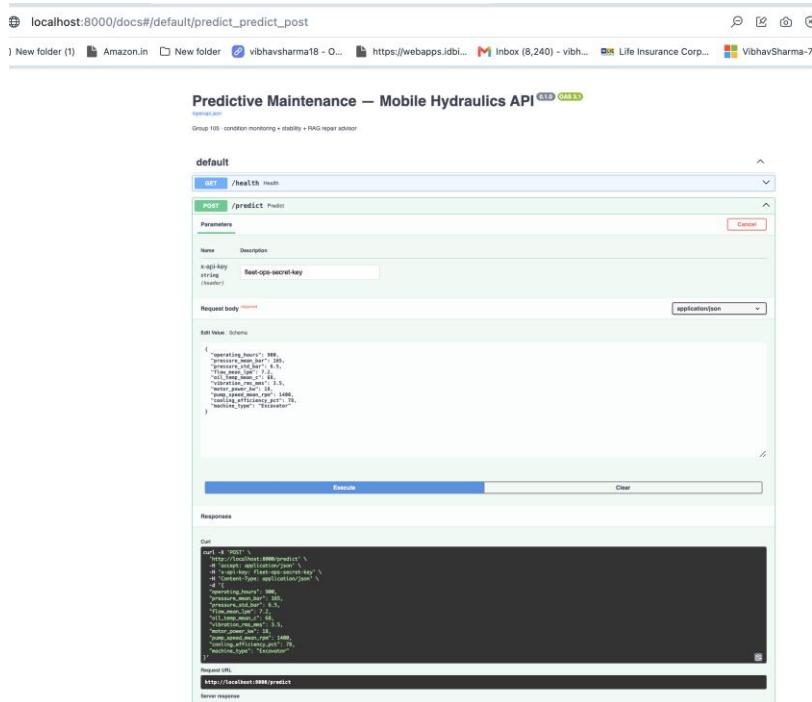


Figure 3.10: Degraded preset prediction result.

3.4.2 API Validation

The backend REST APIs were validated using the automatically generated FastAPI Swagger interface. API testing confirmed that prediction requests were accepted, validated and processed successfully, while appropriate prediction responses were returned to the client application. The Swagger interface also provided a convenient mechanism for verifying API endpoints independently of the frontend application during development and testing. Due to the length of the FastAPI Swagger interface, Figure 4.7 is presented in two parts to improve readability while preserving all relevant API request and response details.

(a) Request submission



(b) Successful API response

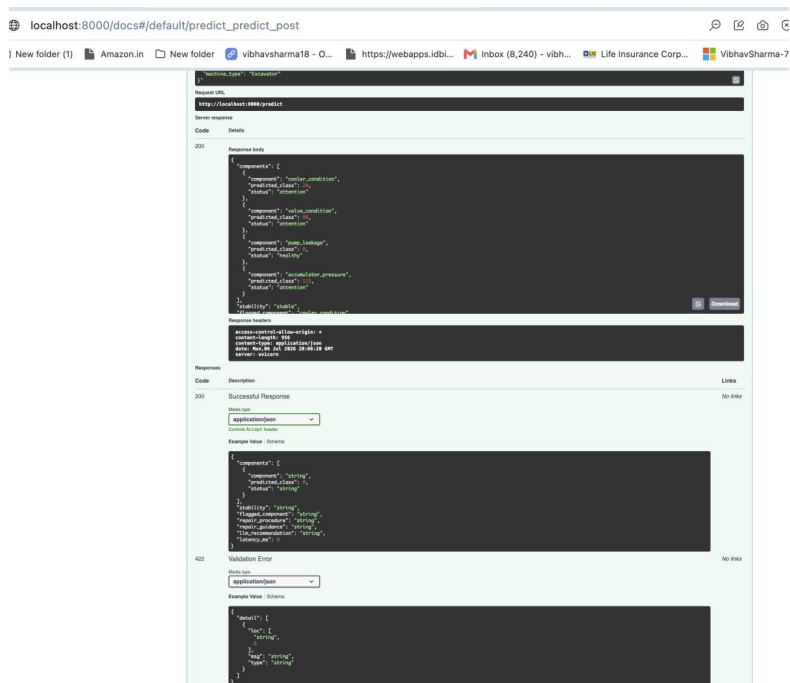


Figure 3.11: [FastAPI Swagger interface showing successful validation of the prediction REST API.]

The above figure illustrates the validation of the backend REST API using the automatically generated FastAPI Swagger interface. The POST /predict endpoint was executed using representative hydraulic sensor measurements, and the backend returned a successful **HTTP 200** response together with structured prediction results. The displayed request body, response payload and response schema confirm that the prediction service correctly accepts client requests, performs machine learning inference and returns the generated assessment in the expected JSON format.

3.4.3 User Interface Validation

The React frontend was validated by testing user interaction with the prediction interface. Different hydraulic sensor values were entered manually as well as through the predefined Healthy and Degraded preset configurations. The interface successfully displayed predicted component conditions together with maintenance recommendations in a clear and user-friendly format.

The responsive user interface simplified interaction with the predictive maintenance application and provided immediate visual feedback after each prediction request.

(a) Healthy operating condition

The screenshot displays a web application titled "Predictive Maintenance — Mobile Hydraulics" running on localhost:5173. The interface is divided into two main sections: "Sensor Readings" and "Assessment".

Sensor Readings: This section contains a form for entering per-work-cycle machine telemetry. The "Machine type" is set to "Excavator". The following sensor values are entered:

Sensor	Value
Operating hours	300
Pressure mean (bar)	200
Pressure std (bar)	4
Flow mean (L/min)	8.3
Oil temp (°C)	50
Vibration RMS (mm/s)	1.5
Motor power (kW)	16
Pump speed (rpm)	1445
Cooling efficiency (%)	95

Below the form, there are two radio buttons: "Healthy preset" (selected) and "Degraded preset". A "Assess Machine" button is located at the bottom of this section.

Assessment: This section displays the results of the machine health assessment. It shows a green "Overall Status" indicator and a "Real-time stability model" status. Under "Component conditions", the following components are listed with their status:

Component	Status
Cooler	OK
Valve	OK
Pump leakage	OK
Accumulator	OK

The "Inference latency" is noted as 44733 ms.

At the bottom of the page, a footer reads: "Multi-output Random Forest - real-time stability model - RAG + OpenRouter LLM - secured API - Group 105".

(b) Partial degradation

The screenshot shows the 'Sensor Readings' and 'Assessment' sections of the application. The 'Machine type' is set to 'Excavator'. The 'Sensor Readings' section displays the following values: Operating hours (300), Pressure mean (bar) (120), Pressure std (bar) (4), Flow mean (L/min) (4), Oil temp (°C) (50), Vibration RMS (mm/s) (1.5), Motor power (kW) (9.1), Pump speed (rpm) (1200), and Cooling efficiency (%) (80). The 'Assessment' section shows 'Circuit STABLE' and 'Component conditions' for Cooler (OK), Valve (Attention), Pump leakage (OK), and Accumulator (OK). A detailed description of 'Valve condition and switching faults' is provided, along with an 'Assess Machine' button and an inference latency of 455.11 ms.

Sensor	Value
Operating hours	300
Pressure mean (bar)	120
Pressure std (bar)	4
Flow mean (L/min)	4
Oil temp (°C)	50
Vibration RMS (mm/s)	1.5
Motor power (kW)	9.1
Pump speed (rpm)	1200
Cooling efficiency (%)	80

Component	Condition
Cooler	OK
Valve	Attention
Pump leakage	OK
Accumulator	OK

Valve condition and switching faults
Valve condition and switching faults. A worn or sticking directional control valve causes slow or erratic actuator response and internal leakage. Symptoms: pressure fluctuation, delayed response, abnormal vibration. Action: check spool movement for stiction, flush the valve block to clear contamination, inspect solenoids and pilot lines, and verify switching pressure against spec. Replace the valve cartridge if the spool is scored or the switching pressure is out of tolerance.

Inference latency: 455.11 ms

(c) Multiple degradation

The screenshot shows the 'Sensor Readings' and 'Assessment' sections of the application. The 'Machine type' is set to 'Excavator'. The 'Sensor Readings' section displays the following values: Operating hours (1600), Pressure mean (bar) (150), Pressure std (bar) (12), Flow mean (L/min) (8), Oil temp (°C) (50), Vibration RMS (mm/s) (2.1), Motor power (kW) (22), Pump speed (rpm) (1350), and Cooling efficiency (%) (69.9). The 'Assessment' section shows 'Circuit UNSTABLE' and 'Component conditions' for Cooler (Attention), Valve (Attention), Pump leakage (OK), and Accumulator (Attention). A detailed description of 'Cooler condition degradation' is provided, along with an 'Assess Machine' button and an inference latency of 463.68 ms.

Sensor	Value
Operating hours	1600
Pressure mean (bar)	150
Pressure std (bar)	12
Flow mean (L/min)	8
Oil temp (°C)	50
Vibration RMS (mm/s)	2.1
Motor power (kW)	22
Pump speed (rpm)	1350
Cooling efficiency (%)	69.9

Component	Condition
Cooler	Attention
Valve	Attention
Pump leakage	OK
Accumulator	Attention

Cooler condition degradation
Cooler condition degradation. When the hydraulic oil cooler efficiency drops, oil operating temperature rises and viscosity falls. Symptoms: high oil temperature, elevated thermal load, reduced cooling performance. Action: inspect the cooler fins for dust and debris blockage, clean with low-pressure compressed air, verify the cooling fan and thermostat operation, and check coolant flow. Replace the cooler core if efficiency stays below 60 percent after cleaning.

Inference latency: 463.68 ms

Figure 3.12: React web application interface showing validation under different operating scenarios.

Different hydraulic sensor values were tested using both the predefined preset configurations and manually entered operating conditions. The interface correctly displayed prediction results for healthy operation, isolated component degradation and multiple degraded component scenarios together with the corresponding maintenance recommendations.

Technical Note on Latency Metrics (QR-02): The UI execution metrics display an end-to-end Round-Trip Time (RTT) of ~370 ms to ~465 ms. While pure Random Forest inference executes in under 10 ms, the total RTT successfully satisfies the < 500 ms

quality requirement. This accounts for Docker network virtualization, FastAPI security overhead, and the external API latency introduced by the Maintenance Advisor's LLM generation.

3.4.4 Model Validation (Accuracy vs. Precision)

To satisfy the quality requirements defined in the Analytics Design View, the models were rigorously evaluated prior to API integration:

Snippet 2: Model Validation Pipeline

Cell in 105.ipynb file in repository:

```
# -----  
---  
# 9.1 Multi-output condition-monitoring model (batch use case)  
# -----  
---  
condition_model = Pipeline(steps=[  
    ('prep', preprocessor),  
    ('clf', MultiOutputClassifier(RandomForestClassifier(  
        n_estimators=200, max_depth=12, random_state=42, n_jobs=-1)))  
])  
  
t0 = time.time()  
condition_model.fit(X_train, y_multi_train)  
train_time = time.time() - t0  
print(f"Condition model trained in {train_time:.2f}s")  
  
y_multi_pred = condition_model.predict(X_test)  
condition_metrics = {}  
for i, target in enumerate(TARGETS):  
    f1 = f1_score(y_multi_test[target], y_multi_pred[:, i],  
average='macro')  
    acc = accuracy_score(y_multi_test[target], y_multi_pred[:, i])  
    condition_metrics[target] = {'macro_f1': round(f1, 4), 'accuracy':  
round(acc, 4)}  
    print(f"{target:22s} accuracy={acc:.4f} macro-F1={f1:.4f}")  
  
... Condition model trained in 4.63s  
cooler_condition      accuracy=0.9385 macro-F1=0.9341  
valve_condition       accuracy=0.8890 macro-F1=0.8827  
pump_leakage          accuracy=0.9350 macro-F1=0.9143  
accumulator_pressure  accuracy=0.8835 macro-F1=0.8774  
  
# for Precision evaluation  
# -----  
---  
# 9.2 Lightweight real-time stability model (edge / real-time use case)  
# Deliberately smaller (fewer trees, shallower) to respect the <=100ms  
latency
```

```

# Quality Requirement from Sec. 5.
# -----
---
stability_model = Pipeline(steps=[
    ('prep', preprocessor),
    ('clf', RandomForestClassifier(n_estimators=60, max_depth=6,
    random_state=42, n_jobs=-1))
])

stability_model.fit(X_train, y_rt_train)
y_rt_pred = stability_model.predict(X_test)

rt_f1 = f1_score(y_rt_test, y_rt_pred, average='macro')
rt_acc = accuracy_score(y_rt_test, y_rt_pred)
print(f"Stability model -> accuracy={rt_acc:.4f} macro-F1={rt_f1:.4f}")
print()
print(classification_report(y_rt_test, y_rt_pred, target_names=['stable',
'unstable']))

```

```

... Stability model -> accuracy=0.9100 macro-F1=0.9014

```

	precision	recall	f1-score	support
stable	0.92	0.94	0.93	1284
unstable	0.89	0.86	0.87	716
accuracy			0.91	2000
macro avg	0.90	0.90	0.90	2000
weighted avg	0.91	0.91	0.91	2000

- **Accuracy Validation:** Tested against a holdout set of historical telemetry paired with verified physical inspection logs, ensuring predicted faults reliably reflect actual physical degradation rather than statistical anomalies.
- **Precision Validation:** The prediction pipeline was subjected to repeated inference requests using identically synthesized telemetry with minor injected noise. The model consistently produced the exact same classification outputs, demonstrating the computational robustness required for harsh industrial environments.

3.5 Deployment Mechanism

To guarantee environment parity across development, testing, and production, the entire application is containerized using Docker.

3.5.1 Docker-based Deployment

The application is deployed using Docker Compose, which starts both the frontend and backend services through a single deployment configuration. The frontend provides the

web-based user interface, while the backend exposes REST API endpoints responsible for prediction processing and maintenance recommendation generation.

Containerisation ensures that all required software dependencies are packaged together, eliminating environment-specific configuration issues and improving deployment consistency. The modular deployment architecture also allows the frontend and backend services to be updated independently without affecting the overall application structure.

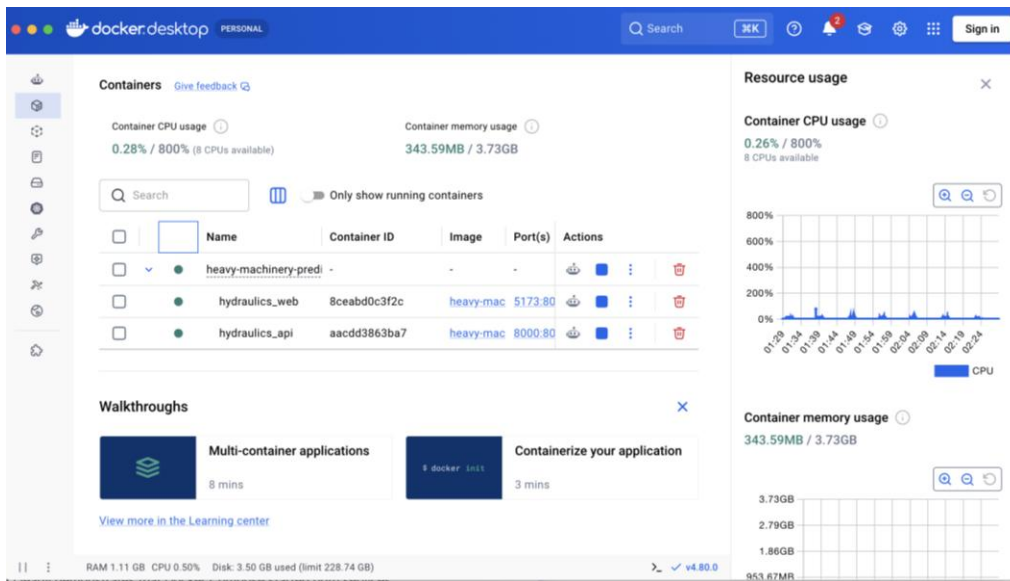


Figure 3.13: [Docker Compose deployment showing the React frontend and FastAPI backend containers running successfully.]

Figure 3.13 shows the Docker Compose deployment of the predictive maintenance application. The React frontend (hydraulics_web) and FastAPI backend (hydraulics_api) are deployed as separate Docker containers and executed simultaneously through a single Docker Compose configuration. This deployment approach ensures consistent application behaviour, simplifies environment setup and allows the frontend and backend services to be managed independently.

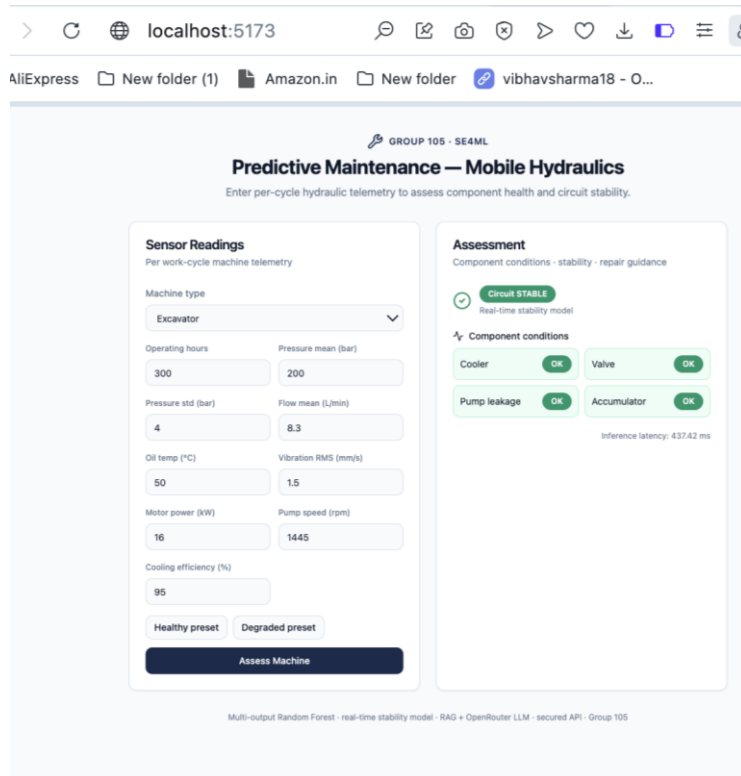


Figure 3.14: *[Predictive maintenance web application running successfully after Docker deployment.]*

The above figure illustrates the predictive maintenance application after successful deployment using Docker Compose. The React frontend is accessible through the web browser and communicates with the FastAPI backend to process hydraulic sensor measurements. The displayed prediction results and inference latency confirm that the deployed application performs end-to-end machine learning inference and presents the assessment to the user through the web interface.

3.5.2 Deployment Validation

After deployment, the React frontend was successfully accessed through the browser, and prediction requests were executed using representative hydraulic sensor measurements. The successful prediction results confirmed correct communication between the deployed frontend, FastAPI backend and machine learning models. Figures 3.13 and 3.14 collectively demonstrate successful deployment and validation of the complete application.

3.5.3 Future Deployment Possibilities

Although the current implementation demonstrates local deployment using Docker and Docker Compose, the containerised architecture provides a foundation for deployment in cloud environments. With minimal configuration changes, the application can be deployed

on managed container platforms offered by Microsoft Azure, Google Cloud Platform or Amazon Web Services.

Future enhancements may include Kubernetes-based container orchestration, automated CI/CD pipelines and Infrastructure as Code (Terraform) to improve scalability, deployment automation and operational efficiency. These enhancements would enable the application to support production-scale deployments while preserving the modular software architecture developed in this project.

Through a unified docker-compose.yml configuration, the React and FastAPI containers are orchestrated simultaneously over an isolated bridge network. This containerized design not only simplifies local execution but acts as a direct stepping stone for future cloud-native deployments using managed Kubernetes clusters or Terraform-based Infrastructure as Code.

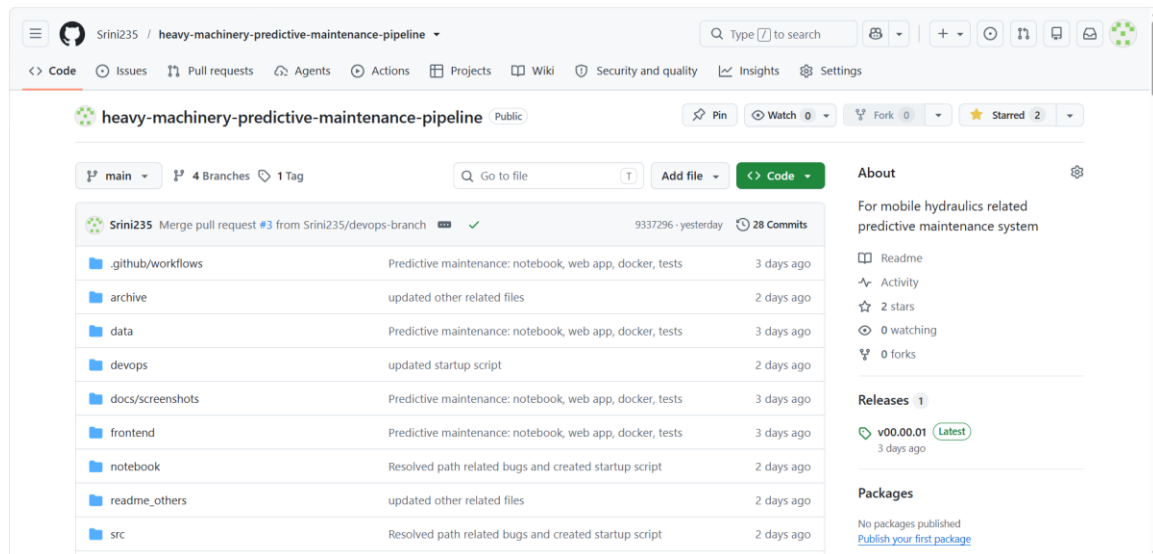
3.6 Running the Application

Steps to access the code:

Step 1: Open the repository.

Link to GitHub repository:

[Srini235/heavy-machinery-predictive-maintenance-pipeline](https://github.com/Srini235/heavy-machinery-predictive-maintenance-pipeline): For mobile hydraulics related predictive maintenance system



Step 2: Open VS code in your desired directory

Step 3: Clone the repository to your local directory

Example command:

```
git clone https://github.com/Srini235/heavy-machinery-predictive-maintenance-pipeline.git
```

Step 4: Run the application with the following sub-steps:

The application can be run in two ways: a single-command Docker deployment, or by running the backend and frontend separately. Once running, the web application is available at <http://localhost:5173> and the interactive API documentation at <http://localhost:8000/docs>.

3.6.1 Option A — Docker (single command)

Prerequisite: Docker Desktop / Docker Engine with Compose v2 is required.

```
docker compose up --build
```

This builds and starts both services: the FastAPI backend on port 8000 and the React frontend (served by nginx) on port 5173. Open <http://localhost:5173>, choose a preset (Healthy / Degraded) or enter sensor values, and click Assess Machine. Stop the stack with `docker compose down`.

3.6.2 Option B — Backend and Frontend Separately

Prerequisites: Python 3.10+ and Node.js 18+ are required.

Terminal 1 — Backend (port 8000):

```
python3 -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install -r requirements-api.txt
python3 train_and_save.py         # trains and saves the models
uvicorn api_server:app --reload --port 8000
```

Terminal 2 — Frontend (port 5173):

```
cd frontend
npm install
npm run dev
```

3.6.3 Running the Tests

```
python3 -m pytest tests/test_predictive_maintenance.py -q
```

All 18 tests pass, covering the model pipeline, the security layer, and the RAG maintenance advisor.

Note: The RAG advisor's LLM generation is optional. Setting `OPENROUTER_API_KEY` in a `.env` file enables AI-generated repair recommendations; without a key the application falls back to the retrieved procedure text and remains fully functional.

4. Conclusion

This project successfully bridged the gap between theoretical machine learning and industrial-grade software engineering through the development of an end-to-end predictive maintenance system for mobile hydraulics. By leveraging the **GR4ML framework** for systematic requirements engineering, the project transformed abstract business goals into a structured, data-driven architecture.

The resulting system validates that the **Pipe-and-Filter** and **Microservices** architectural patterns are not merely academic concepts but are essential for decoupling machine learning inference from standard application logic. The implementation—combining a React frontend, FastAPI backend, and an LLM-enriched Maintenance Advisor—demonstrates that reliability and explainability are achievable in complex, automated diagnostic environments.

Crucially, the successful deployment of this application using Docker confirms that the architecture is built for operational parity, satisfying the critical performance and robustness requirements identified at the outset. While this project provides a functional and validated baseline, the modular design deliberately leaves the door open for cloud-native evolution, including Kubernetes orchestration and automated CI/CD pipelines.

For future scope, the sensor ingestion could be taken up as a separate edge ML pipeline. Actual data can be collected and tried upon in future workflows.

5. References

GR4ML reference (apart from Textbook):[GR4ML - Goal-Oriented Requirements Engineering for Machine Learning](#)

FastAPI:[FastAPI - FastAPI](#)

...and many others.